The difference is illustrated by the following code:

```
a = (11..20).collect { |i| (i % 4 == 0)..(i % 3 == 0) ? i : nil }
a         # => [nil, 12, nil, nil, nil, 16, 17, 18, nil, 20]

a = (11..20).collect { |i| (i % 4 == 0)...(i % 3 == 0) ? i : nil }
a         # => [nil, 12, 13, 14, 15, 16, 17, 18, nil, 20]
```

Regular Expressions in Boolean Expressions

If Ruby has been invoked via the -e parameter in the command line, and only in that case, a regular expression by itself without any other operator in a boolean expression matches against the current value of the variable $_, the most recent line input by gets or readline:

```
$ ruby -ne 'print if /one/' testfile
This is line one
```

In regular code, the use of implicit operands and $_ has been largely phased out, so it is better to use an explicit match against a variable.

if and unless Expressions

if *boolean-expression* ‹then›
  body
‹elsif *boolean-expression* ‹then›
  body ›*
‹ else
  body ›
end

unless *boolean-expression* ‹then›
  body
‹else
  body ›
end

The then keyword separates the body from the condition, and is not required if the body starts on a new line. The value of an if or unless expression is the value of the last expression evaluated in whichever body is executed.

if and unless Modifiers

expression if *boolean-expression*
expression unless *boolean-expression*

This evaluates *expression* only if *boolean-expression* is true (for if) or false (for unless).

Ternary Operator

boolean-expression ? *expr1* : *expr2*

This returns *expr1* if *boolean-expression* is true and *expr2* otherwise.

case Expressions

Ruby has two forms of conditional case statements. The first allows a series of conditions to be evaluated, executing code corresponding to the first condition that is true:

case
when ‹*boolean-expression*›+ ‹then›
  body
when ‹*boolean-expression*›+ ‹then›
  body